

TaxPal

Description

TaxPal serves as a comprehensive Personal Finance & Tax Estimator platform that bridges the gap between income generation and financial administration. By leveraging the **MERN** (MongoDB, Express.js, React.js, Node.js) technology stack, the application provides a seamless, responsive, and secure environment for users to manage their fiscal responsibilities. The platform automates the often-tedious process of tax calculation based on user-defined income slabs, tracks real-time budgetary constraints to prevent overspending, and generates detailed financial reports for record-keeping.

The primary goal of TaxPal is to democratize access to sophisticated financial tools, allowing freelancers and small business owners to focus on their core work rather than getting bogged down by administrative financial tasks. Through features like the Tax Calendar and Budget Maintenance, TaxPal ensures that users remain compliant with tax regulations while maintaining a healthy financial status.

Purpose

The purpose of TaxPal is to simplify and enhance the resource-matching process for both businesses and freelancers by providing a centralized platform for all financial activities. The specific objectives include:

- **Optimizing Tax Management:** To automate the calculation of quarterly and annual tax liabilities, thereby reducing the manual effort and high error rate associated with traditional spreadsheet-based methods.
- **Improving Financial Awareness:** To provide users with immediate, transparent insights into their financial health, including real-time tracking of income versus expenses and savings rates.
- **Enhancing Budget Control:** To empower individuals to set strict spending limits across various categories (e.g., Food, Rent, Transport) and receive visual feedback when approaching these limits.
- **Streamlining Processes:** To replace time-intensive, manual hiring and tracking workflows with a smart, technology-driven approach that encourages efficiency.

By addressing inefficiencies in traditional resource allocation and financial tracking, TaxPal aims to become the go-to platform for short-term hiring and project-based financial collaboration.

Getting Started

Prerequisites

Before setting up and running the application, ensure the following software and tools are installed on your system:

- * Node.js and npm
 - * Download and install from the Node.js official website.
 - * npm comes bundled with Node.js.
- * MongoDB
 - * Install and set up MongoDB locally or use a cloud-hosted solution like MongoDB Atlas.
- * IDE
 - * Use a development environment like Visual Studio Code for code editing and debugging.

Installation

Follow these steps to set up the project:

1. Clone the Repository

```
```bash
git clone <repository_url>
cd Taxpal_Group1
```
```

2. Install Dependencies

```
* Backend:
```bash
cd backend
npm install
```

* Frontend:
```bash
cd frontend
npm install
```
```

Environment Setup for TaxPal

To run the project seamlessly, you need to set up environment variables for both the backend and frontend. Create a `.env` file in the root directory of the backend project and add the required configurations.

Required Environment Variables

| Variable Name | Description | Example Value |
|-----------------|--|----------------------------------|
| MONGO_URI | Connection string for the MongoDB database. | mongodb://localhost:27017/taxpal |
| JWT_SECRET | Secret key for signing JWT tokens. | your_super_secret_key |
| PORT | Port for the backend server. | 5000 |
| EMAIL_USER | Email address used for sending OTPs/notifications. | your-email@gmail.com |
| EMAIL_PASS | Password or app-specific password for the email. | your-email-password |
| FRONTEND_ORIGIN | URL of the frontend application (for CORS). | http://localhost:5173 |

Example .env Configuration

Here's an example `.env` file for your backend:

```

```.env
MONGO_URI=mongodb://localhost:27017/taxpal
JWT_SECRET=supersecretkey123

```

```
PORT=5000
EMAIL_USER=your-email@gmail.com
EMAIL_PASS=your-email-password
FRONTEND_ORIGIN=http://localhost:5173
````
```

Steps to Set Up the Environment

1. Create a `.env` file in the root of your `backend` directory.
2. Copy the example configuration above and paste it into the `.env` file.
3. Replace placeholder values with your actual configuration.

Running the Application

1. Start the Backend

- * Navigate to the backend directory:

```
``bash
cd backend
````
```
- \* Run the backend server:

```
``bash
npm start
````
```

(Or `npm run dev` for development with nodemon)

2. Start the Frontend

- * Navigate to the frontend directory:

```
``bash
cd frontend
````
```
- \* Run the development server:

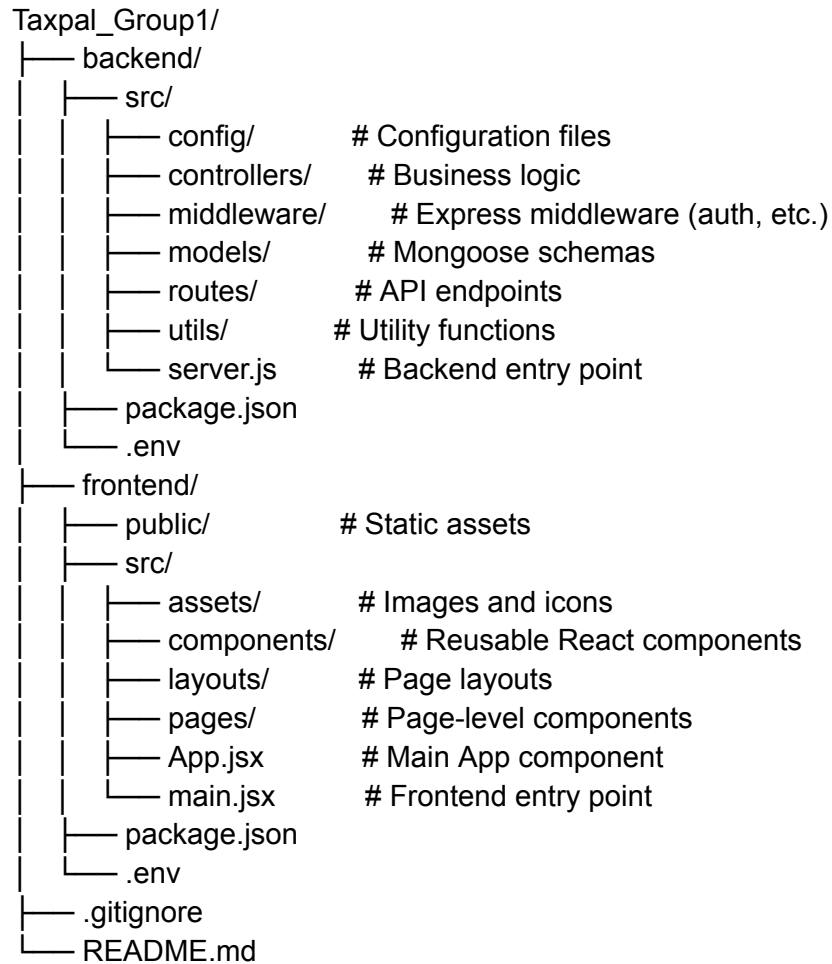
```
``bash
npm run dev
````
```

Accessing the Application

- * Frontend: Visit `http://localhost:5173` (or the port specified by Vite).
- * Backend API: Accessible at `http://localhost:5000` (or the port specified in `.env`).

Folder Structure

A well-organized folder structure helps maintain clarity and efficiency. Below is the folder structure for TaxPal.



Key Features

User Authentication

Secure user registration and login using JWT (JSON Web Tokens). Includes email-based OTP verification for signup and password resets.

Dashboard

A comprehensive dashboard that provides a snapshot of the user's financial status, including recent transactions, budget status, and notifications.

Transaction Management

Users can log income and expenses, categorize them, and view a history of all transactions.

Budgeting

Allows users to set monthly budgets for different categories. Visual indicators (Good, Warning, Over) help users track their spending against these budgets.

Reports

Generates visual reports and analytics to help users understand their spending habits over time.

Tax Estimation

Provides tools to estimate tax liabilities based on recorded income and expenses, aiding in financial planning.

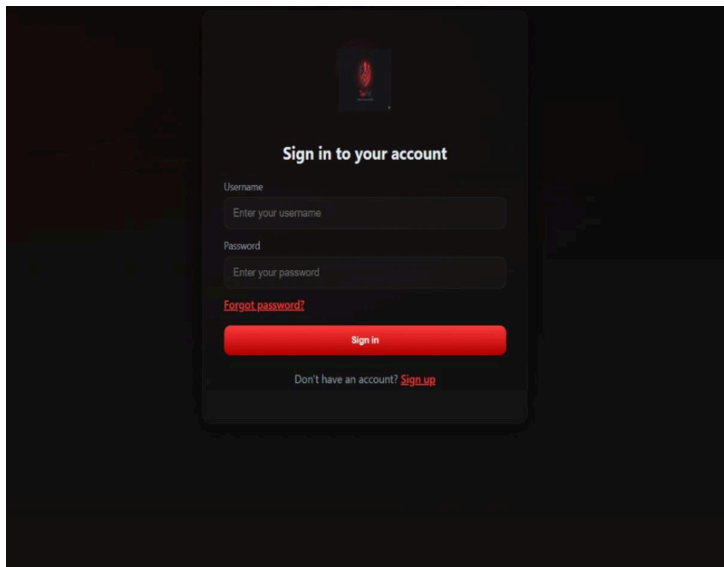
Project Components

Login Component

Description: Manages the user login process.

Key Features:

- * Form Validation: Ensures valid email and password format.
- * Secure Authentication: content-type application/json; communicates with the backend to verify credentials and store the JWT.



Register Component

Description: Provides signup functionality for new users.

Key Features:

- * OTP Verification: a specific verification step using email OTP to confirm identity.
- * Input Validation: Checks for strong passwords and valid email addresses.

Create your account

Enter your details to get started with TaxPal

Username

Password

Full name (optional)

Email address

Country of residence

Select your country

Income range (optional)

Select income range

Verify email & continue

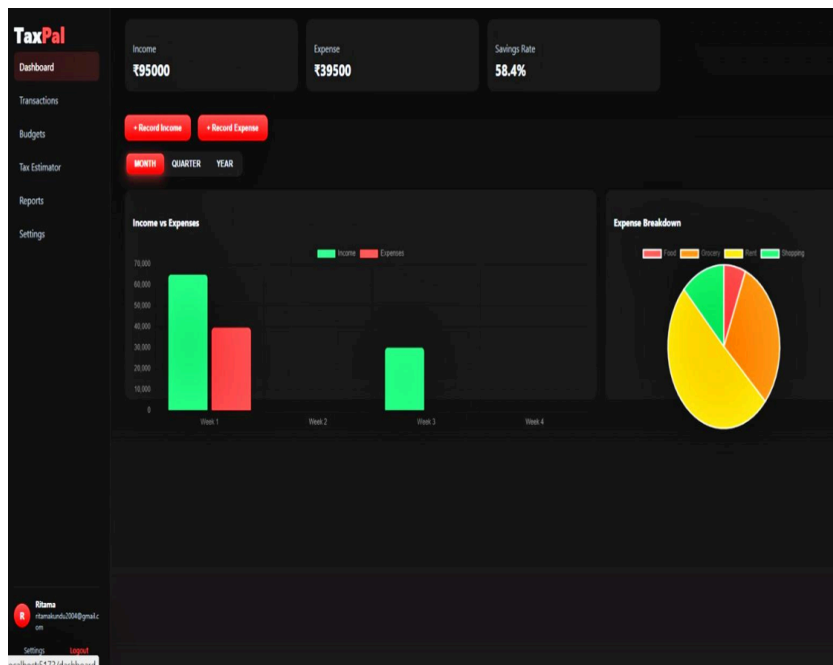
Already registered? [Sign in](#)

Dashboard Component

Description: The main landing page after login.

Key Features:

- * Summary Cards: Displays total income, expenses, and balance.
- * Recent Activity :Shows the latest transactions.

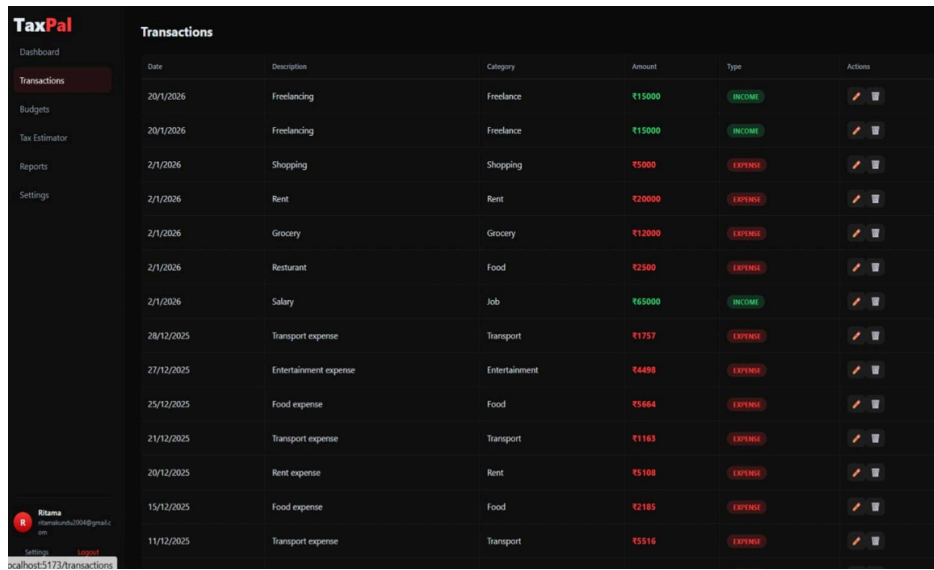


Transactions Component

Description: Interface for managing financial records.

Key Features:

- * Add Transaction: Form to input amount, date, category, and description.
- * List View: filters and sorts transactions.



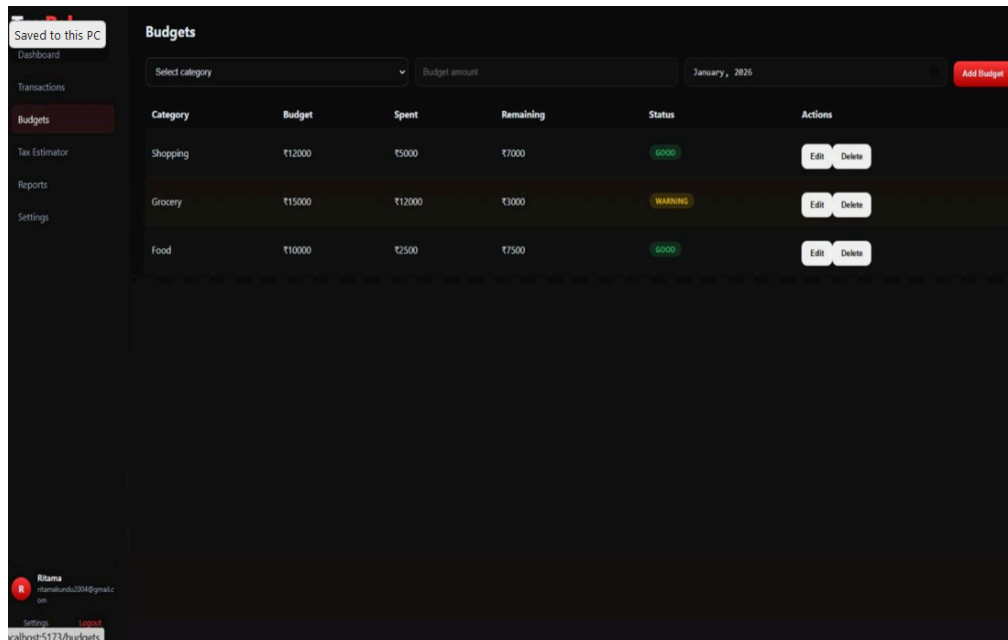
| Date | Description | Category | Amount | Type | Actions |
|------------|-----------------------|---------------|--------|---------|---|
| 20/1/2026 | Freelancing | Freelance | ₹15000 | INCOME |   |
| 20/1/2026 | Freelancing | Freelance | ₹15000 | INCOME |   |
| 2/1/2026 | Shopping | Shopping | ₹5000 | EXPENSE |   |
| 2/1/2026 | Rent | Rent | ₹20000 | EXPENSE |   |
| 2/1/2026 | Grocery | Grocery | ₹12000 | EXPENSE |   |
| 2/1/2026 | Restaurant | Food | ₹2500 | EXPENSE |   |
| 2/1/2026 | Salary | Job | ₹65000 | INCOME |   |
| 28/12/2025 | Transport expense | Transport | ₹1757 | EXPENSE |   |
| 27/12/2025 | Entertainment expense | Entertainment | ₹4498 | EXPENSE |   |
| 25/12/2025 | Food expense | Food | ₹5664 | EXPENSE |   |
| 21/12/2025 | Transport expense | Transport | ₹1163 | EXPENSE |   |
| 20/12/2025 | Rent expense | Rent | ₹5108 | EXPENSE |   |
| 15/12/2025 | Food expense | Food | ₹2185 | EXPENSE |   |
| 11/12/2025 | Transport expense | Transport | ₹5516 | EXPENSE |   |

Budgets Component

Description: Manage monthly spending limits.

Key Features:

- * Create Budget: Set amounts for specific categories.
- * Progress Bars: Visual representation of spent vs. remaining budget.



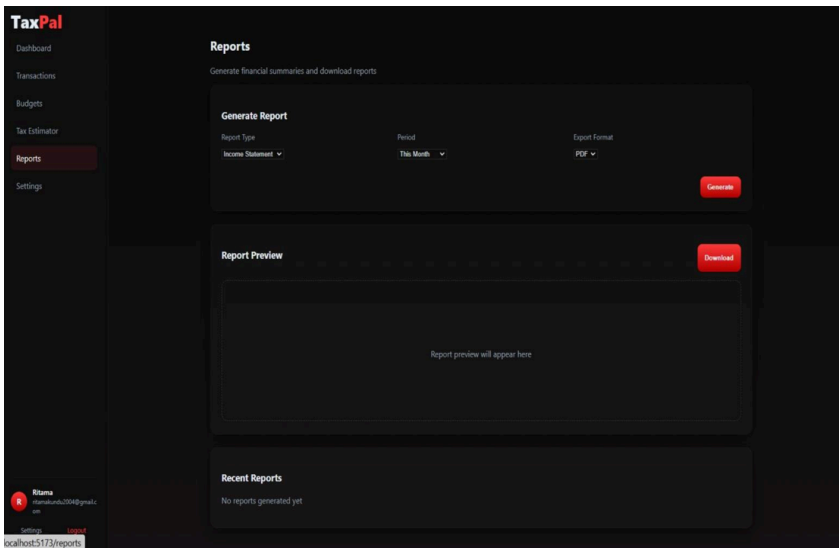
| Category | Budget | Spent | Remaining | Status | Actions |
|----------|--------|--------|-----------|---------|--|
| Shopping | ₹12000 | ₹5000 | ₹7000 | GOOD |   |
| Grocery | ₹15000 | ₹12000 | ₹3000 | WARNING |   |
| Food | ₹10000 | ₹2500 | ₹7500 | GOOD |   |

Reports Component

Description: Visual analytics of financial data.

Key Features:

- * Charts: Pie charts and bar graphs using Chart.js to visualize expense distribution.
- * Charts: Pie charts and bar graphs using Chart.js to visualize expense distribution.

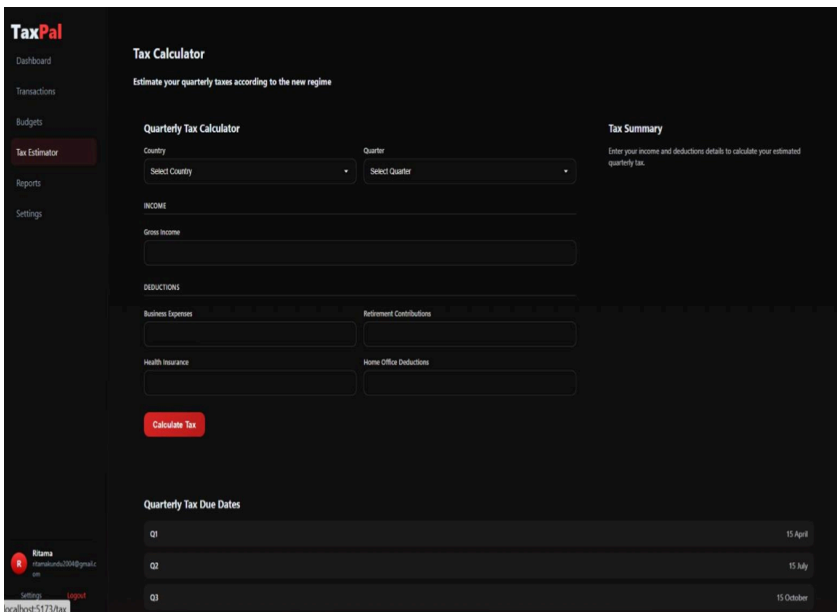


Tax Estimator Component

Description: A tool for calculating estimated taxes.

Key Features:

- * Input Fields: Allows users to input tax-relevant data.
- * Calculation Logic: Applies tax rules to estimate liability.
- * Calculation Logic: Applies tax rules to estimate liability.

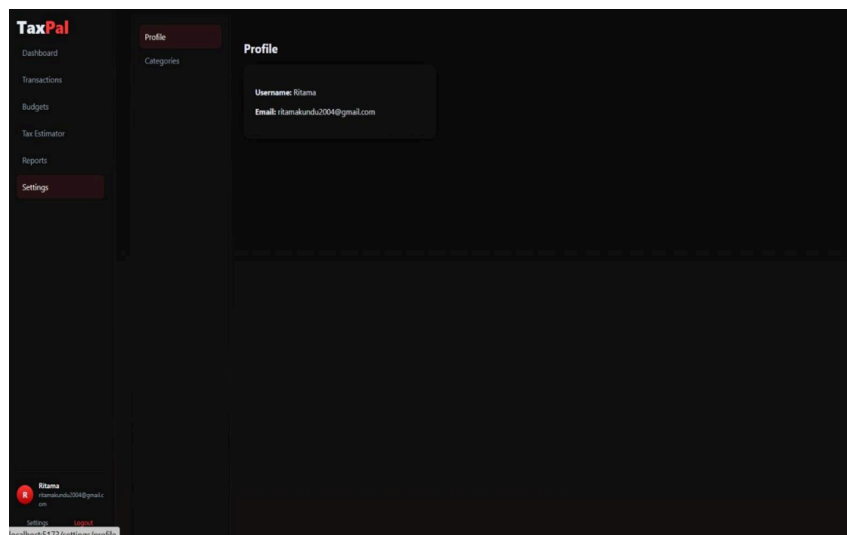


Settings Component

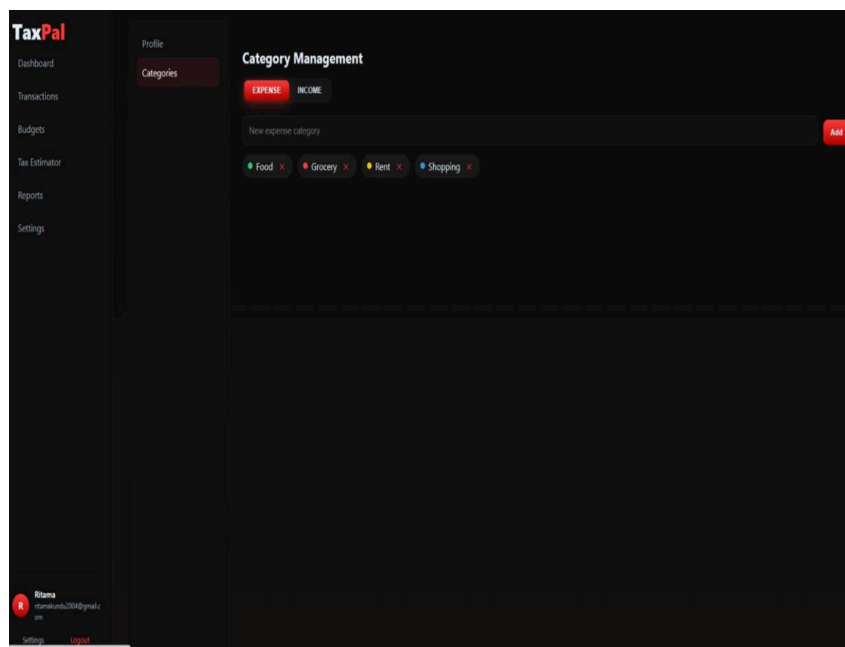
Description: Manage user profile and application settings.

Key Features:

*Profile: Update user information such as name and email.



*Categories: Manage income and expense categories (add, delete, customize tags).



API Components

User Authentication & Management

Signup

- * Route: POST /api/auth/signup`
- * Description: Register a new user and send OTP.
- * Input:

```
```json
{
 "name": "John Doe",
 "email": "john@example.com",
 "password": "Password123"
}
```
```
- * Output:

```
`200 OK`: `{ "message": "OTP sent to email" }`
```

Verify Signup OTP

- *Route: POST /api/auth/signup-verify`
- *Description: Verify OTP to complete registration.
- *Input:

```
```json
{
 "email": "john@example.com",
 "otp": "123456"
}
```
```
- *Output:

```
200 OK`: `{ "message": "Signup successful", "token": "..." }`
```

Login

- *Route: POST /api/auth/login`
- *Description: Authenticate a user.
- *Input:

```
```json
{
 "email": "john@example.com",
 "password": "Password123"
}
```
```
- * **Output:**
 - * `200 OK`: `{ "token": "...", "user": { ... } }`

Transactions

Get All Transactions

*Route: GET /api/transactions`

*Description: Retrieve all transactions for the logged-in user.

*Input: None (Headers: `Authorization: Bearer <token>`)

*Output:

200 OK: `[ { "id": "...", "type": "expense", "amount": 100, ... }, ... ]`

Create Transaction

*Route: POST /api/transactions`

*Description: Add a new income or expense.

*Input:

```
```json
{
 "type": "expense",
 "description": "Groceries",
 "amount": 50,
 "category": "Food",
 "date": "2023-10-27"
}
...`
```

\*Output:

\*201 Created: `{ "id": "...", ... }`

### Delete Transaction

\*Route: DELETE /api/transactions/:id`

\*Description: Delete a specific transaction.

\*Input: URL Parameter `id`

\*Output:

200 OK: `{ "success": true }`

### Budgets

#### Get Budgets

\*Route: GET /api/budgets`

\*Description: Retrieve all budgets with calculated spent/remaining amounts.

\*Output:

200 OK: `[ { "category": "Food", "amount": 500, "spent": 150, "status": "good" }, ... ]`

#### Create Budget

\*Route: `POST /api/budgets`

\*Input:

```
```json
{
  "category": "Food",
  ...
}
```

```
"amount": 500,  
"month": "2023-10"  
}  
...
```

* Output:

```
`201 Created`: `{ ... }`
```

Error Handling and Troubleshooting

Database Connection Failures

Error: `MongoDB connection error`

Troubleshooting:

- * Check if MongoDB is running locally (`mongod`) or if your Atlas cluster is accessible.
- * Verify the `MONGO\_URI` in the `.env` file is correct.

SMTP Not Working

Error: Emails (OTP) not being sent.

Troubleshooting:

- * Ensure `EMAIL\_USER` and `EMAIL\_PASS` are correct in `.env`.
- * If using Gmail, ensure "Less secure app access" or an App Password is used.

Port Conflicts

Error: `EADDRINUSE: address already in use`

Troubleshooting:

- * Change the `PORT` in `.env`.
- * Kill the process using the port (e.g., `npx kill-port 5000`).

Maintenance and Support

Backing Up the Database

- * Use `mongodump` or MongoDB Atlas backup features to regularly back up user data.

Updates

- * Pull the latest changes from the repository: `git pull origin main`.
- * Install new dependencies: `npm install`.

Glossary

* JWT (JSON Web Token): A compact, URL-safe means of representing claims to be transferred between two parties. Used here for secure authentication.

* OTP (One-Time Password): A password that is valid for only one login session or transaction, used for enhanced security.

* MERN Stack :The technology stack used: MongoDB, Express.js, React, Node.js.

Future Enhancements

- * Mobile App: Develop a native mobile application using React Native.
- * Bank Linking: Integration with banking APIs (like Plaid) to automatically import transactions.
- * AI Insights: Use AI to provide personalized financial advice based on spending habits.